USN No: 23BTRCL014
Name: DHARUN P

Experiment 3: Real-Time Stream Processing using Apache Spark Structured Streaming

Topic

Real-Time Stream Processing with Apache Spark Structured Streaming: local-mode installation on Windows, the micro-batch and unbounded-table execution model, the high-level DataFrame API, Kafka integration, a live word-frequency demonstration, and an event-time windowed analytics case study — contrasted throughout with the Apache Storm topology built in Experiment 2.

Objective

To install and configure Apache Spark 3.5.x in local mode on Windows (Java 17, Python, and the Hadoop winutils shim), to understand the Structured Streaming execution model — the unbounded input table, micro-batch processing, output modes, triggers, event time, windowing and watermarks — and to build and run live Kafka→Spark streaming jobs using the high-level DataFrame API. The central learning outcome is to show that the same continuous, stateful word-count computation built with Storm in Experiment 2 can be expressed in a small fraction of the code, and that harder operations such as event-time windowing come essentially for free.

Introduction to Spark Structured Streaming

Apache Spark is a unified engine for large-scale data processing. Structured Streaming is its modern streaming API, built on the same DataFrame/SQL interface used for batch data. Where Experiment 1 showed Kafka moving records reliably from producers to consumers, and Experiment 2 showed Storm continuously processing those records through a topology of spouts and bolts, Structured Streaming processes the same continuous data using ordinary table queries — and needs far less code to do it.

The central idea is that a stream is treated as an unbounded input table that keeps growing as new records arrive. The developer writes a query against this table exactly as they would against a static table; Spark is responsible for re-running that query incrementally as new data arrives and keeping the result up to date. This is the opposite of Storm's model, where the developer must explicitly describe how data flows from component to component and must manage state by hand.

Execution happens in micro-batches. On each trigger, Spark takes whatever new rows have arrived, runs one small batch, updates the result, and repeats. This differs from Storm's record-at-a-time processing: Storm offers slightly lower latency, but Spark gives batch-style simplicity, automatic state management, event-time windowing, and exactly-once fault tolerance through checkpointing — with a fraction of the code.

Every Structured Streaming job has the same three-part shape: readStream (define the source), a set of DataFrame transformations (identical to batch code), and writeStream.start() (define the sink, the output mode, and the trigger). An output mode states what to emit each batch: complete re-emits the whole result table, append emits only new final rows, and update emits only rows that changed. This experiment reuses the same Kafka broker and Zookeeper from Experiments 1 and 2, so the streaming source is identical — only the processing engine changes.

SET 1: Prerequisites and Installing Apache Spark

Goal: Confirm Java 17 and the Experiment 1 Kafka/Zookeeper environment are available, then install Python, Spark 3.5.9, and the Windows Hadoop shim (winutils), and verify the installation.

Task 1 — Verify Java 17 and confirm Kafka/Zookeeper are available

Spark 3.5.x supports Java 8, 11, or 17. Java 17 is confirmed installed. Kafka and Zookeeper from Experiment 1 are also confirmed present, and Python 3.13.2 is available on the system.

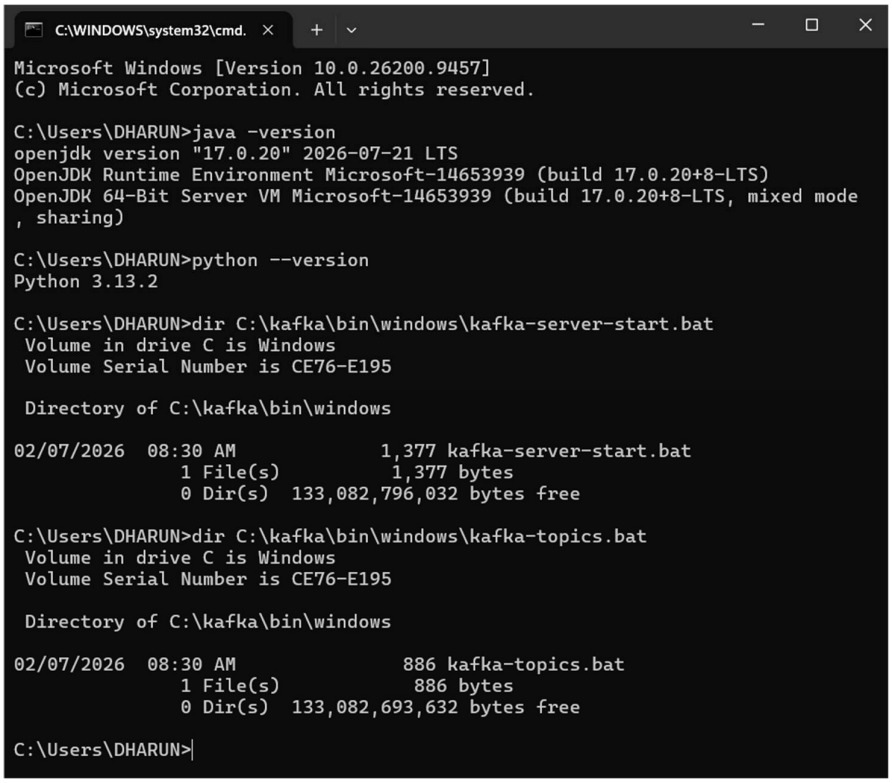
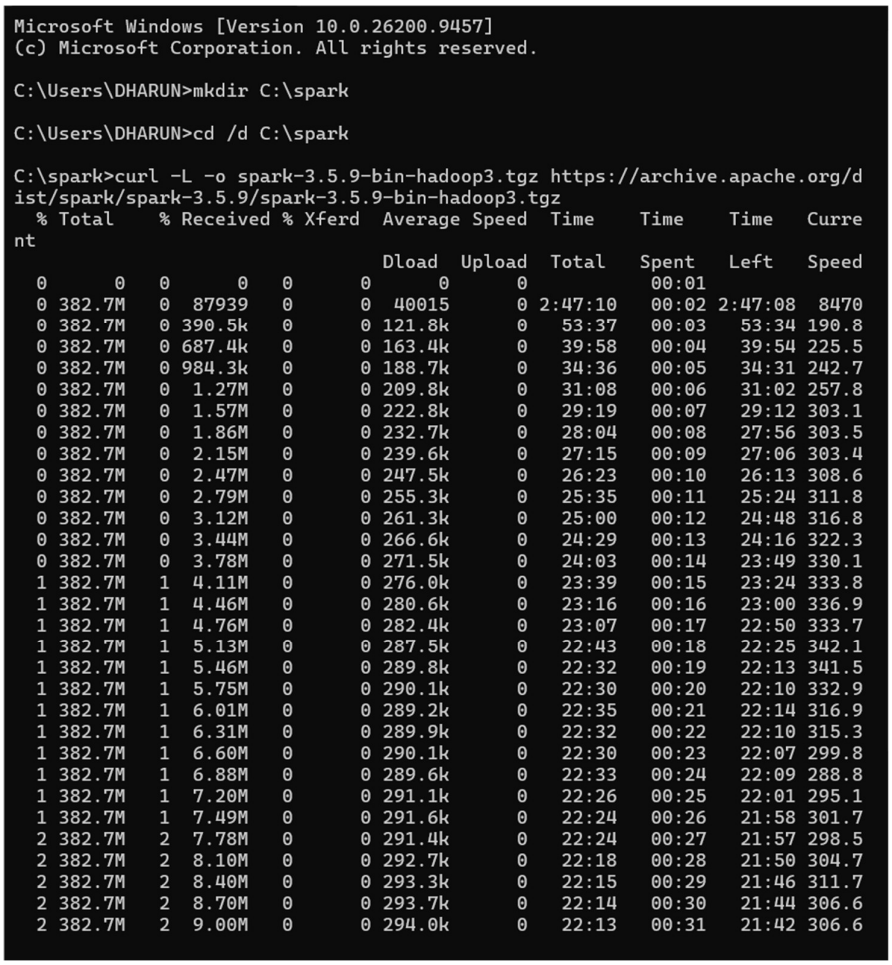


Fig 1.1 — java -version, python --version, and Kafka batch files confirmed present

Task 2 — Download and extract Apache Spark 3.5.9

Spark release 3.5.9 (Pre-built for Apache Hadoop 3.3 and later) was downloaded as a .tgz archive and extracted into C:\spark.



```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DHARUN>mkdir C:\spark

C:\Users\DHARUN>cd /d C:\spark

C:\spark>curl -L -o spark-3.5.9-bin-hadoop3.tgz https://archive.apache.org/d
ist/spark/spark-3.5.9/spark-3.5.9-bin-hadoop3.tgz
  % Total    % Received % Xferd  Average Speed  Time    Time    Time   Curre
nt
                                 Dload  Upload  Total   Spent   Left   Speed
  0      0   0      0   0      0      0       0           00:01
  0 382.7M   0  87939   0      0  40015      0 2:47:10   00:02 2:47:08  8470
  0 382.7M   0 390.5k   0      0 121.8k      0   53:37   00:03   53:34 190.8
  0 382.7M   0 687.4k   0      0 163.4k      0   39:58   00:04   39:54 225.5
  0 382.7M   0 984.3k   0      0 188.7k      0   34:36   00:05   34:31 242.7
  0 382.7M   0  1.27M   0      0 209.8k      0   31:08   00:06   31:02 257.8
  0 382.7M   0  1.57M   0      0 222.8k      0   29:19   00:07   29:12 303.1
  0 382.7M   0  1.86M   0      0 232.7k      0   28:04   00:08   27:56 303.5
  0 382.7M   0  2.15M   0      0 239.6k      0   27:15   00:09   27:06 303.4
  0 382.7M   0  2.47M   0      0 247.5k      0   26:23   00:10   26:13 308.6
  0 382.7M   0  2.79M   0      0 255.3k      0   25:35   00:11   25:24 311.8
  0 382.7M   0  3.12M   0      0 261.3k      0   25:00   00:12   24:48 316.8
  0 382.7M   0  3.44M   0      0 266.6k      0   24:29   00:13   24:16 322.3
  0 382.7M   0  3.78M   0      0 271.5k      0   24:03   00:14   23:49 330.1
  1 382.7M   1  4.11M   0      0 276.0k      0   23:39   00:15   23:24 333.8
  1 382.7M   1  4.46M   0      0 280.6k      0   23:16   00:16   23:00 336.9
  1 382.7M   1  4.76M   0      0 282.4k      0   23:07   00:17   22:50 333.7
  1 382.7M   1  5.13M   0      0 287.5k      0   22:43   00:18   22:25 342.1
  1 382.7M   1  5.46M   0      0 289.8k      0   22:32   00:19   22:13 341.5
  1 382.7M   1  5.75M   0      0 290.1k      0   22:30   00:20   22:10 332.9
  1 382.7M   1  6.01M   0      0 289.2k      0   22:35   00:21   22:14 316.9
  1 382.7M   1  6.31M   0      0 289.9k      0   22:32   00:22   22:10 315.3
  1 382.7M   1  6.60M   0      0 290.1k      0   22:30   00:23   22:07 299.8
  1 382.7M   1  6.88M   0      0 289.6k      0   22:33   00:24   22:09 288.8
  1 382.7M   1  7.20M   0      0 291.1k      0   22:26   00:25   22:01 295.1
  1 382.7M   1  7.49M   0      0 291.6k      0   22:24   00:26   21:58 301.7
  2 382.7M   2  7.78M   0      0 291.4k      0   22:24   00:27   21:57 298.5
  2 382.7M   2  8.10M   0      0 292.7k      0   22:18   00:28   21:50 304.7
  2 382.7M   2  8.40M   0      0 293.3k      0   22:15   00:29   21:46 311.7
  2 382.7M   2  8.70M   0      0 293.7k      0   22:14   00:30   21:44 306.6
  2 382.7M   2  9.00M   0      0 294.0k      0   22:13   00:31   21:42 306.6
```

Fig 1.2 — Downloading spark-3.5.9-bin-hadoop3.tgz with curl

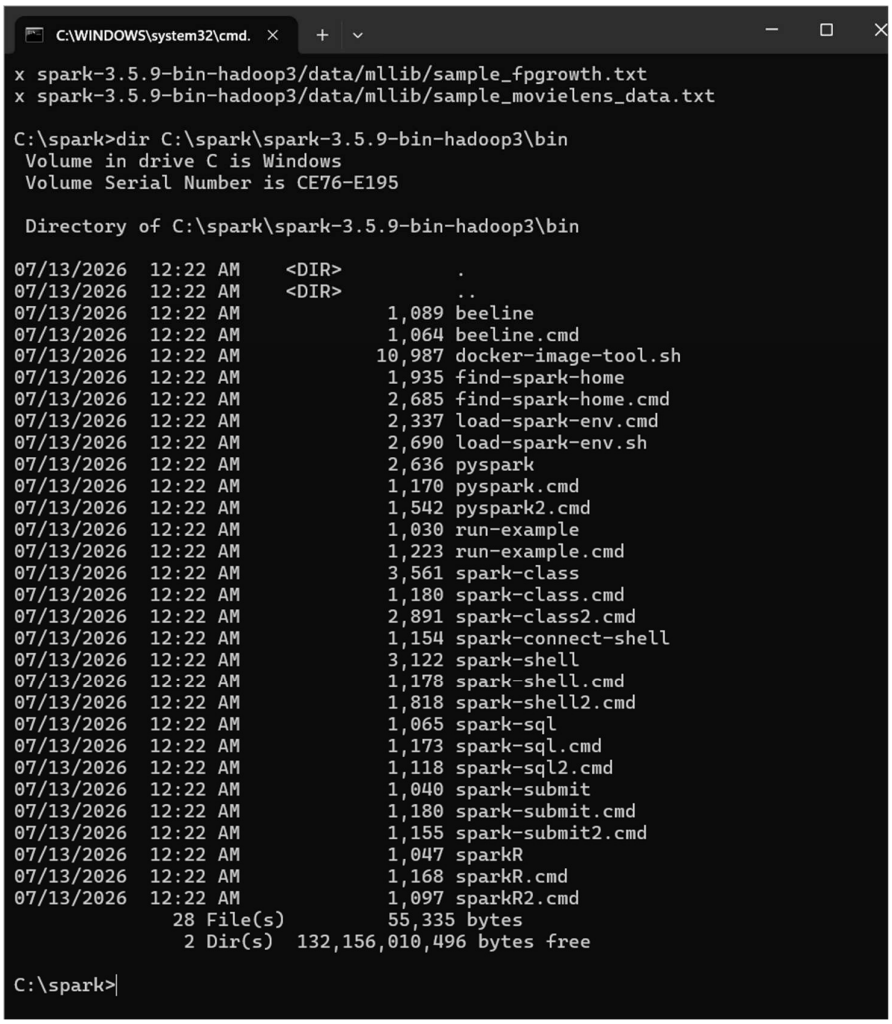


Fig 1.3 — Archive extracted; launcher scripts confirmed in C:\spark\spark-3.5.9-bin-hadoop3\bin

Task 3 — Install winutils (the Windows Hadoop shim)

winutils.exe and hadoop.dll (matching the Hadoop 3.3.6 line bundled with Spark) were downloaded into C:\hadoop\bin so Spark's file-system layer can perform disk operations such as checkpointing on Windows.

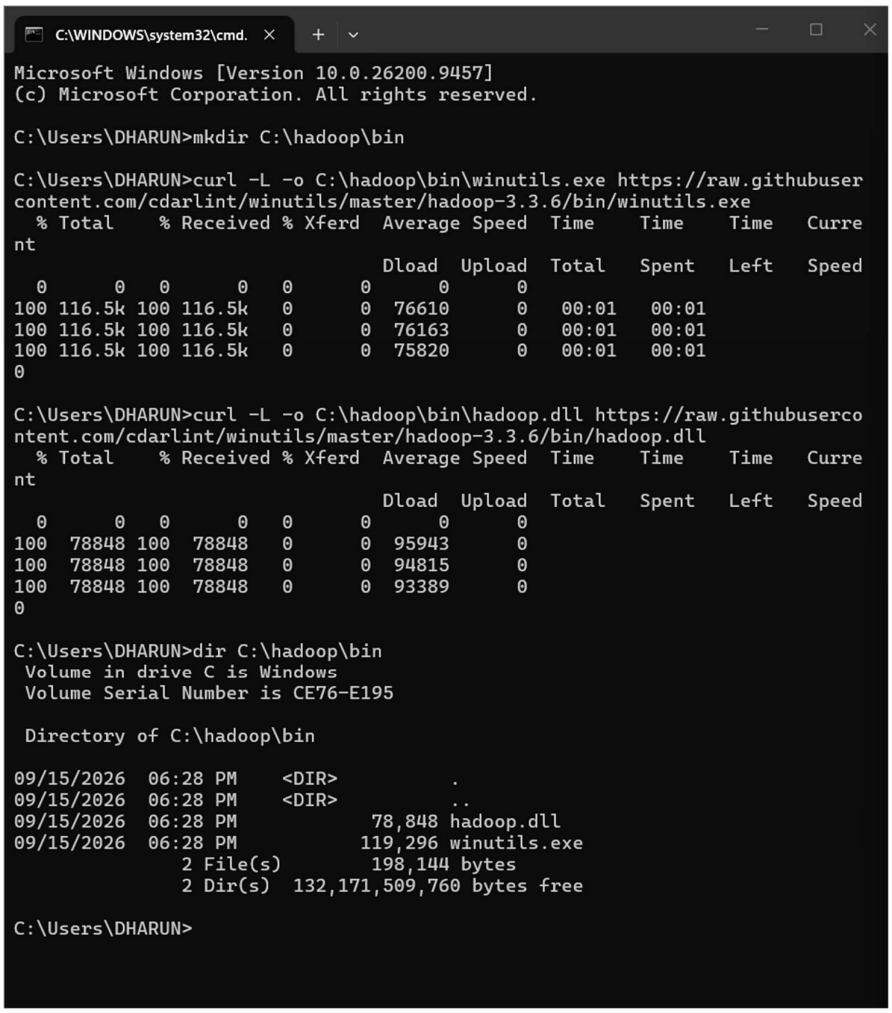


Fig 1.4 — winutils.exe and hadoop.dll downloaded into C:\hadoop\bin

Task 4 — Set environment variables

JAVA_HOME, SPARK_HOME, HADOOP_HOME, PYSPARK_PYTHON, and PYSPARK_DRIVER_PYTHON were set as system variables, and the Java/Spark/Hadoop bin folders were added to Path.



Fig 1.5 — JAVA_HOME, SPARK_HOME, HADOOP_HOME, PYSPARK_PYTHON, and PYSPARK_DRIVER_PYTHON set with setx

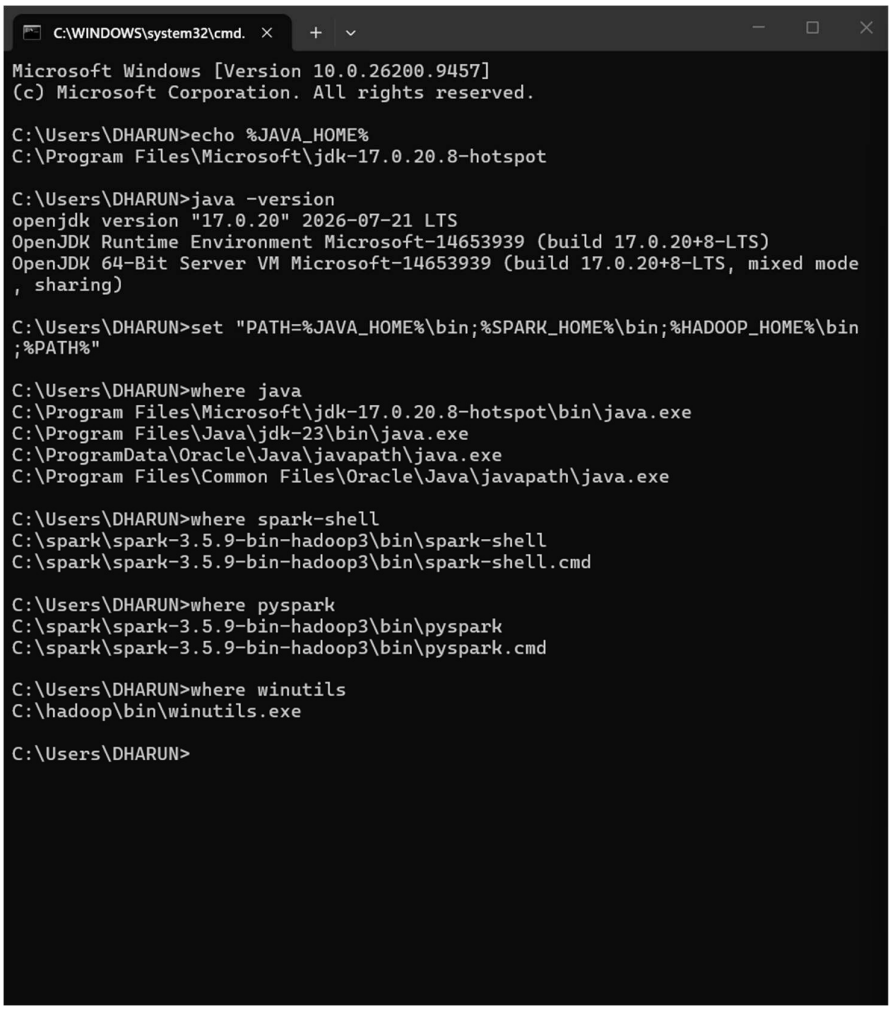


Fig 1.6 — Fresh terminal confirms JAVA_HOME, java -version (17), and where java / spark-shell / pyspark / winutils all resolve correctly

Task 5 — Verify the Spark installation

The Scala shell (spark-shell) was launched and a smoke-test job (spark.range(5).show()) was run to confirm the full Java + Spark + Hadoop-shim chain is correctly configured.



Fig 1.7 — spark-shell launches on Spark 3.5.9 / Scala 2.12.18 / Java 17.0.20; spark.range(5).show() prints a five-row table

Result: Spark 3.5.9 runs on Java 17 with winutils in place; the smoke test computes and prints a five-row table, confirming the full Java + Spark + Hadoop-shim chain is correctly configured.

SET 2: The Structured Streaming Model — First Streaming Job

Goal: Make the micro-batch model visible with a source that needs no external services, then run a socket word-count to see the output modes in action — all in a handful of lines of the DataFrame API.

Task 1 — The rate source

Code used (rate_demo.py):

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("RateDemo").getOrCreate()
spark.sparkContext.setLogLevel("WARN")

df = spark.readStream.format("rate").option("rowsPerSecond", 5).load()
query = df.writeStream.format("console").outputMode("append").start()
query.awaitTermination()
```

The built-in rate source generates rows at a fixed pace with no Kafka required. Running the job produced a new micro-batch roughly every second, each containing 5 new rows with an auto-generated timestamp and an incrementing value — making the micro-batch execution model directly visible.



Fig 2.1 — Rate source streaming query producing Batch 0 through Batch 3, five rows per batch

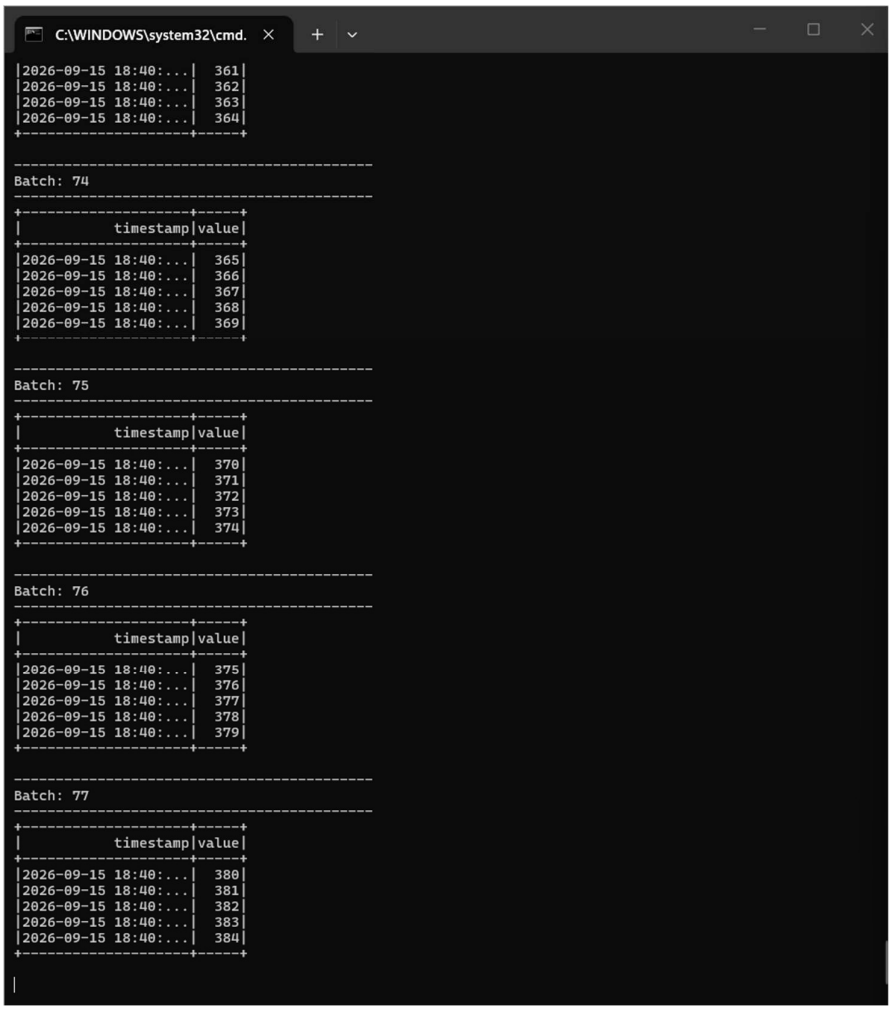


Fig 2.2 — The query continues producing batches (Batch 74-77 shown), confirming continuous micro-batch execution over time

Task 2 & 3 — Socket word-count and output modes

A lightweight Python line server (in place of netcat, unavailable on Windows) was started on port 9999 to feed sentences to Spark. In a second terminal, a socket word-count query was run using outputMode("complete").

Code used (socket word-count, run interactively in pyspark):

```
lines = spark.readStream.format("socket") \
    .option("host", "localhost").option("port", 9999).load()
words = lines.select(explode(split(lines.value, " ")).alias("word"))
wordCounts = words.groupBy("word").count()
query = wordCounts.writeStream.outputMode("complete").format("console").start()
```

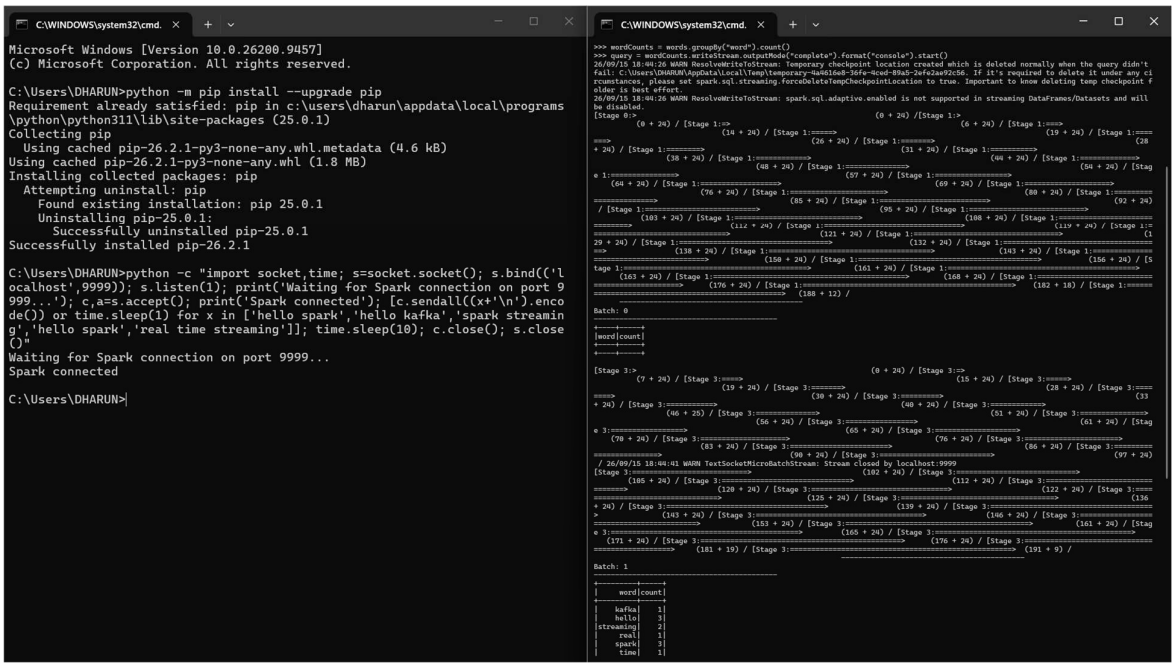


Fig 2.3 - Left: Python socket server sending test sentences on port 9999. Right: socket word-count query starting and printing running counts

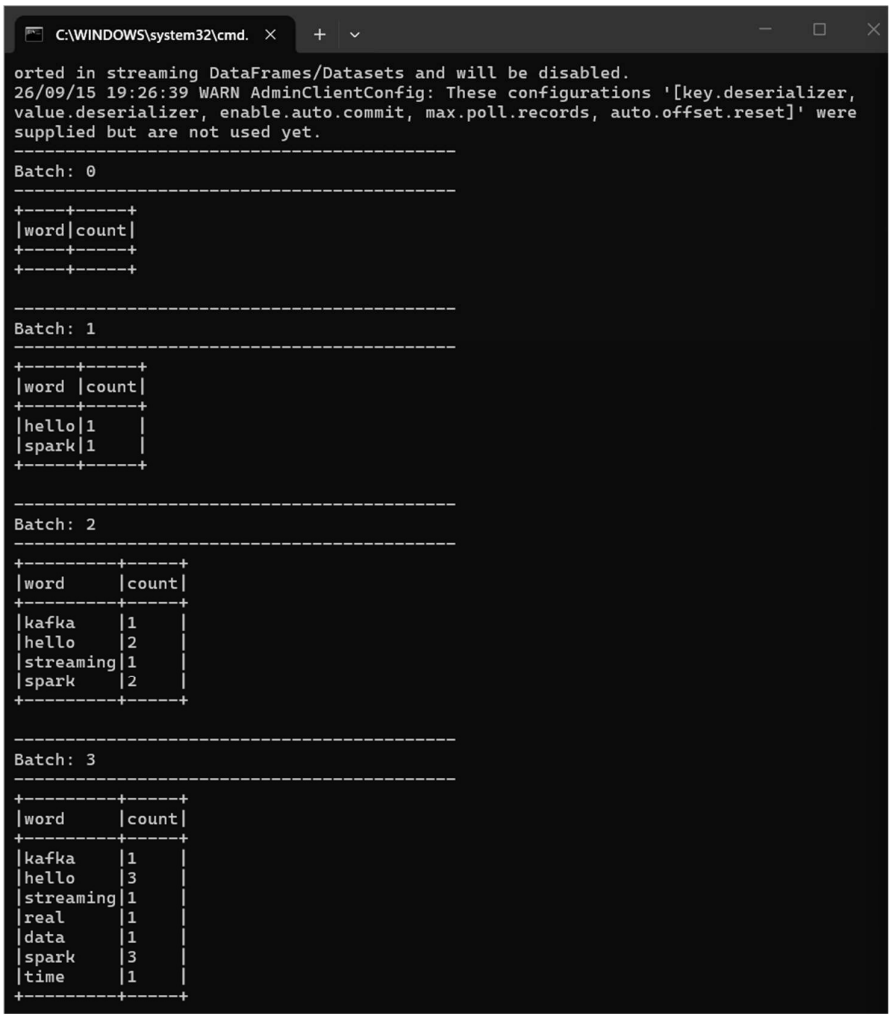


Fig 2.4 — Successive batches of the socket word-count: counts accumulate cumulatively across Batch 0-3 under outputMode("complete")

Result: The streaming word-count required only a handful of lines of DataFrame code with no manual state and no topology wiring — compared with the multi-class Storm topology of Experiment 2. Running the query under outputMode("complete") shows the full running result table on every batch, illustrating Structured Streaming's declarative output-mode model.

SET 3: Building the Kafka–Spark Word-Count Job

Goal: Replace the socket source with the Experiment 1 Kafka topic, so Spark consumes the same live stream that Storm did in Experiment 2 — using the Kafka source connector and the DataFrame API.

Task 1 — Verify Kafka and Zookeeper, and the topic

Zookeeper and the Kafka broker were confirmed running (QuorumPeerMain and Kafka processes visible via jps, broker listening on port 9092), and the existing first-topic from Experiment 1 was confirmed via kafka-topics --list and --describe.



Fig 3.1 — Zookeeper server starting up and initializing successfully

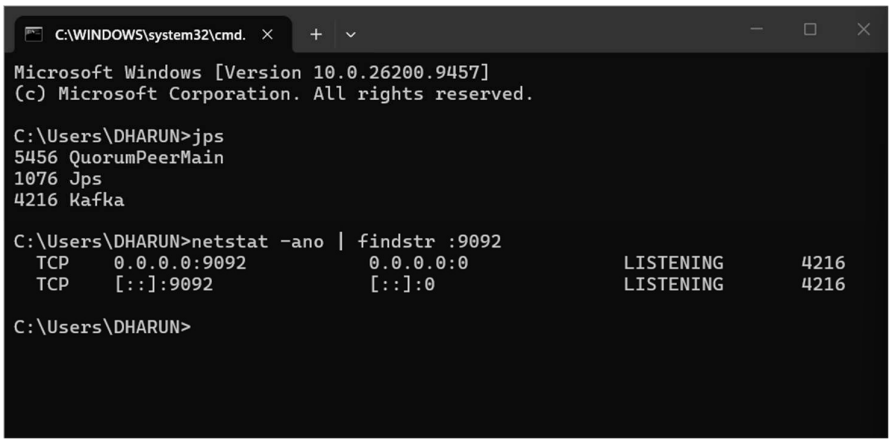


Fig 3.2 — jps confirms QuorumPeerMain and Kafka processes running; netstat confirms the broker listening on port 9092

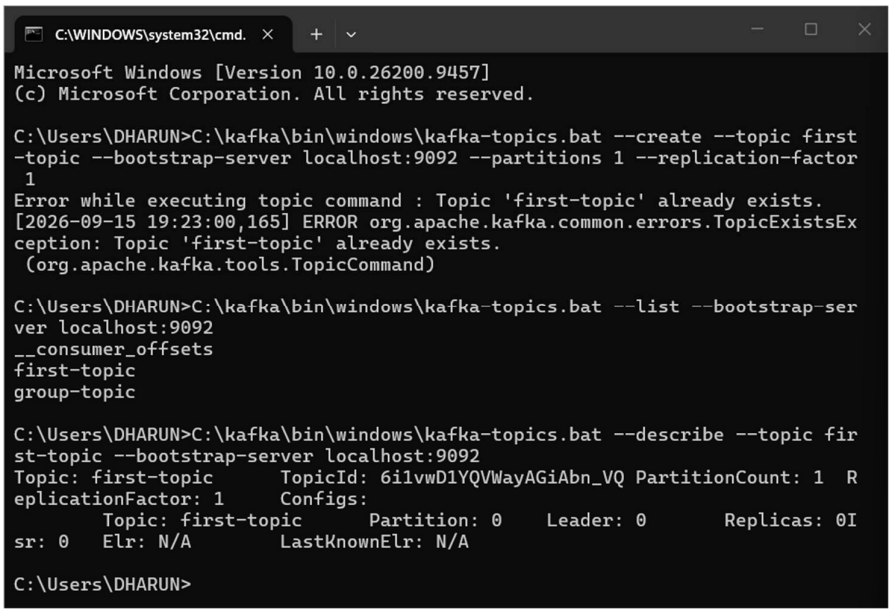


Fig 3.3 — first-topic confirmed present via kafka-topics --list and --describe

Task 2 — The Kafka word-count job

Code used (kafka_wordcount.py):

```
raw = spark.readStream.format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "first-topic") \
    .option("startingOffsets", "latest").load()

words = raw.selectExpr("CAST(value AS STRING) AS line") \
    .select(explode(split(col("line"), " ")).alias("word"))
counts = words.groupBy("word").count()

query = counts.writeStream.outputMode("complete") \
    .format("console").start()
```

The job was launched with spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.9, consuming from the same first-topic used by Storm in Experiment 2.



Fig 3.4 — Kafka word-count job starting up (Batch 0, empty — no messages produced yet)

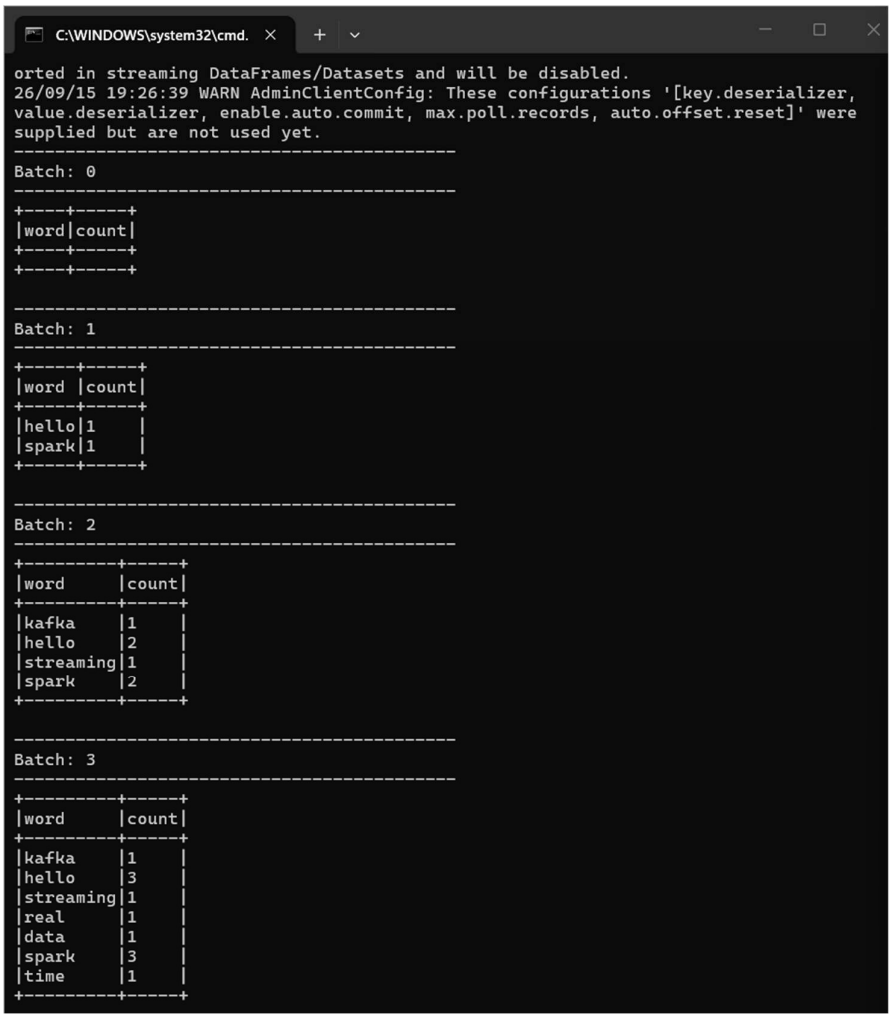


Fig 3.5 — Running counts for words such as spark, hello, kafka, streaming, real, data, and time across successive batches as Kafka messages arrive

Result: Spark consumed the same Kafka topic that Storm consumed in Experiment 2 and produced the same live, stateful word-frequency result — but the whole job is a single short Python file with no build step, no jar, and no manually managed count state.

SET 4: Case Study — Event-Time Windowed Analytics (Unique Signature Task)

Goal: Move beyond word count to a realistic pipeline that computes revenue per product in 30-second event-time windows, with a watermark for late data — an operation that is trivial in Structured Streaming but hard to hand-build in Storm.

Task 1 — Create the orders topic and a JSON order producer

A new Kafka topic orders was created, the kafka-python client was installed, and an order_producer.py script was run to emit realistic order events as JSON, each carrying its own event_time.

```
producer = KafkaProducer(
    bootstrap_servers="localhost:9092",
    value_serializer=lambda v: json.dumps(v).encode("utf-8"))

event = {
    "order_id": random.randint(1000, 9999),
    "product": random.choice(products),
    "amount": round(random.uniform(5, 500), 2),
    "event_time": datetime.datetime.now().isoformat()
```

```
}
producer.send("orders", event)
```

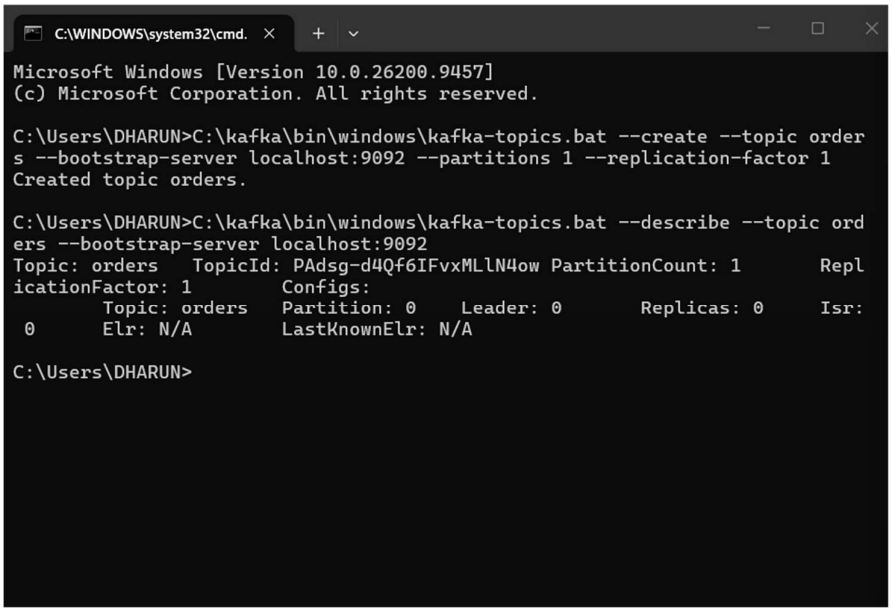


Fig 4.1 — orders topic created and described via kafka-topics

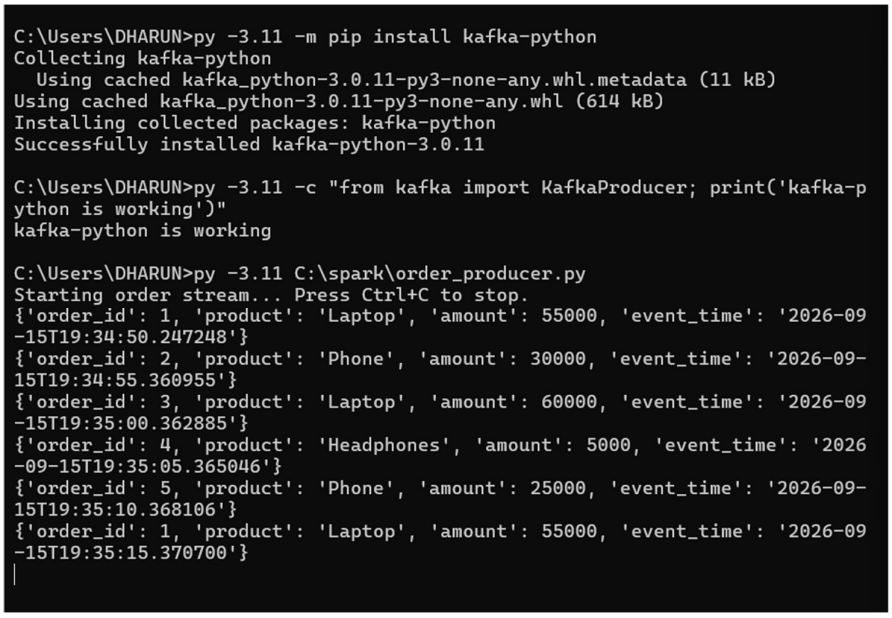


Fig 4.2 — kafka-python installed; order_producer.py streaming JSON order events (order_id, product, amount, event_time) to the orders topic

Task 2 — The windowed analytics job

Code used (orders_analytics.py): the value column is parsed from JSON with an explicit schema, a watermark of one minute bounds how long late records are accepted, and a 30-second tumbling window groups revenue by product.

```
schema = StructType([
    StructField("order_id", IntegerType()),
    StructField("product", StringType()),
    StructField("amount", DoubleType()),
    StructField("event_time", TimestampType())
])

orders = raw.select(
```

```
    from_json(col("value").cast("string"), schema).alias("data")
).select("data.*")

windowed = orders.withWatermark("event_time", "1 minute") \
    .groupBy(
        window(col("event_time"), "30 seconds"),
        col("product")
    ) \
    .agg(
        sum("amount").alias("total_revenue"),
        count("order_id").alias("order_count")
    )

query = windowed.writeStream \
    .outputMode("update") \
    .format("console") \
    .option("truncate", "false") \
    .start()
```



Fig 4.3 — Kafka orders stream, JSON schema, and windowed aggregation defined and started interactively in pyspark

Task 3 — Observe live windows



Fig 4.4 — Batch 0 of the windowed query: per-30-second-window, per-product total_revenue and order_count

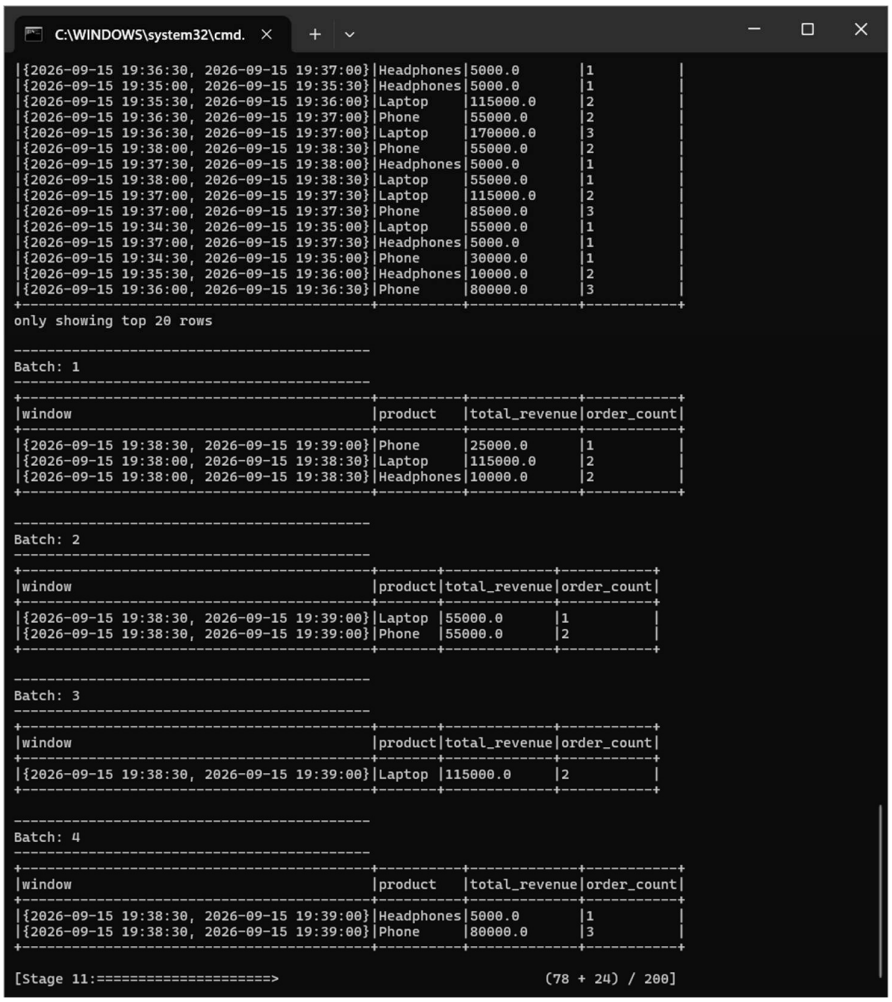


Fig 4.5 — Successive batches (Batch 1-4) showing new 30-second windows appearing as more orders arrive

Result: Per-window, per-product revenue was computed continuously on event time, with the watermark bounding how long the query waits for late data before finalizing a window's state. Achieving the same event-time windowing in Storm would require tick tuples or Trident and substantial manual state code; here it is two extra lines (withWatermark and window). This is the clearest illustration of what the high-level DataFrame API buys over a low-level topology.

SET 5: Output Modes, Checkpointing, and the Storm Contrast (Signature Task)

Goal: Demonstrate fault-tolerant, stateful recovery via checkpointing — the analogue of Storm's managed lifecycle — and quantify how much less code Spark needed for the same results.

Task 1 — Add a checkpoint location and run the job

A checkpoint location lets a query recover its state and Kafka offsets after a restart. checkpoint_wordcount.py adds a checkpointLocation option to the writeStream of the Kafka word-count job, and is launched via spark-submit with the Kafka connector package. The Path / PySpark environment variables needed to be re-applied in the fresh terminal before spark-submit was recognized.



Fig 5.1 — checkpoint_wordcount.py; spark-submit initially not recognized, resolved after re-applying PATH, then confirmed via where spark-submit

Task 2 — Checkpointed job output

With checkpointLocation set, the job was run against the Kafka first-topic. Running word counts (kafka, hello, streaming, real, data, spark, time) are printed in Batch 0, with Spark maintaining state under the configured checkpoint directory to support exactly-once recovery on restart.



Fig 5.2 — Checkpointed Kafka word-count job running; Batch 0 word counts printed with checkpointing enabled

Task 3 — Quantify the contrast with Storm (Experiment 2)

Aspect	Apache Storm (Exp 2)	Spark Structured Streaming (Exp 3)
Programming model	Spouts, bolts, tuples, groupings wired into a topology	A DataFrame query over an unbounded table
Word-count code	~4 Java classes, 100+ lines, Maven build	~8 lines of Python, no build step
State	Manual (HashMap inside a bolt)	Automatic managed state store
Event-time windows	Manual (tick tuples / Trident)	Built-in window() + watermark
Fault tolerance	At-least-once via acking	Exactly-once via checkpointing
Deployment	storm jar to a running cluster	spark-submit

Result: The same continuous, stateful stream computation — and a windowed analytic that Storm cannot express without significant extra machinery — was built in a fraction of the code, with state and exactly-once recovery handled by the engine. This directly demonstrates the value of Spark's high-level, micro-batch DataFrame API over a low-level record-at-a-time topology.

Overall Conclusion

Apache Spark 3.5.9 was installed in local mode on Windows on top of the Java 17, Zookeeper, and Kafka environment from Experiments 1 and 2, including the winutils Hadoop shim required on Windows. The

Structured Streaming model — unbounded input table, micro-batch execution, output modes, triggers, event time, windowing, and watermarks — was demonstrated through a progression of jobs: a rate source to make micro-batches visible, a socket word-count to show the output modes, a Kafka word-count consuming the same topic that Storm used, and an event-time windowed revenue analytic. Checkpointing was used to show stateful restart recovery. Throughout, the same computations built with a multi-class Storm topology in Experiment 2 were expressed in a fraction of the code, with state, windowing, and fault tolerance provided by the engine rather than by the developer. Together, Experiments 1, 2, and 3 cover the full modern streaming pipeline — reliable partitioned messaging (Kafka), low-level topology processing (Storm), and high-level declarative stream processing (Spark) — directly supporting CO1 and CO2 of the course.